CS219 Project1 Report

12411723 杨嘉乐

目录 (Table of Contents)

1. 前言 (Introduction)
2. 核心特性与功能展示 (Core Features & Showcase)
    - 2.1 运行与构建环境
    - 2.2 交互体验
    - 2.3 数值格式
    - 2.4 错误诊断
    - 2.5 数学语法规范
    - 2.6 无损高精度与动态截断
    - 2.7 I/O 与科学计数法输出
    - 2.8 其他函数的实现
3. 系统架构与算法实现 (System Architecture & Algorithms)
    - 3.1 模块化分层设计
    - 3.2 大数存储模型：十亿进制压位
    - 3.3 加减法逻辑：位元对齐与进位路由
    - 3.4 乘法引擎：常规 O(N^2) 与 FFT O(N log N)
    - 3.5 除法：二分查找与原位试商优化
    - 3.6 递归下降语法解析器
    - 3.7 其他函数
4. 内存管理与核心性能优化 (Memory Management & Core Performance Optimization)
    - 4.1 堆内存的生命周期约束
    - 4.2 十亿进制（Base-10^9）压位
    - 4.3 零分配原位计算

- 4.4 代码规范重构与告警消除
- 4.5 性能与内存测试数据评估

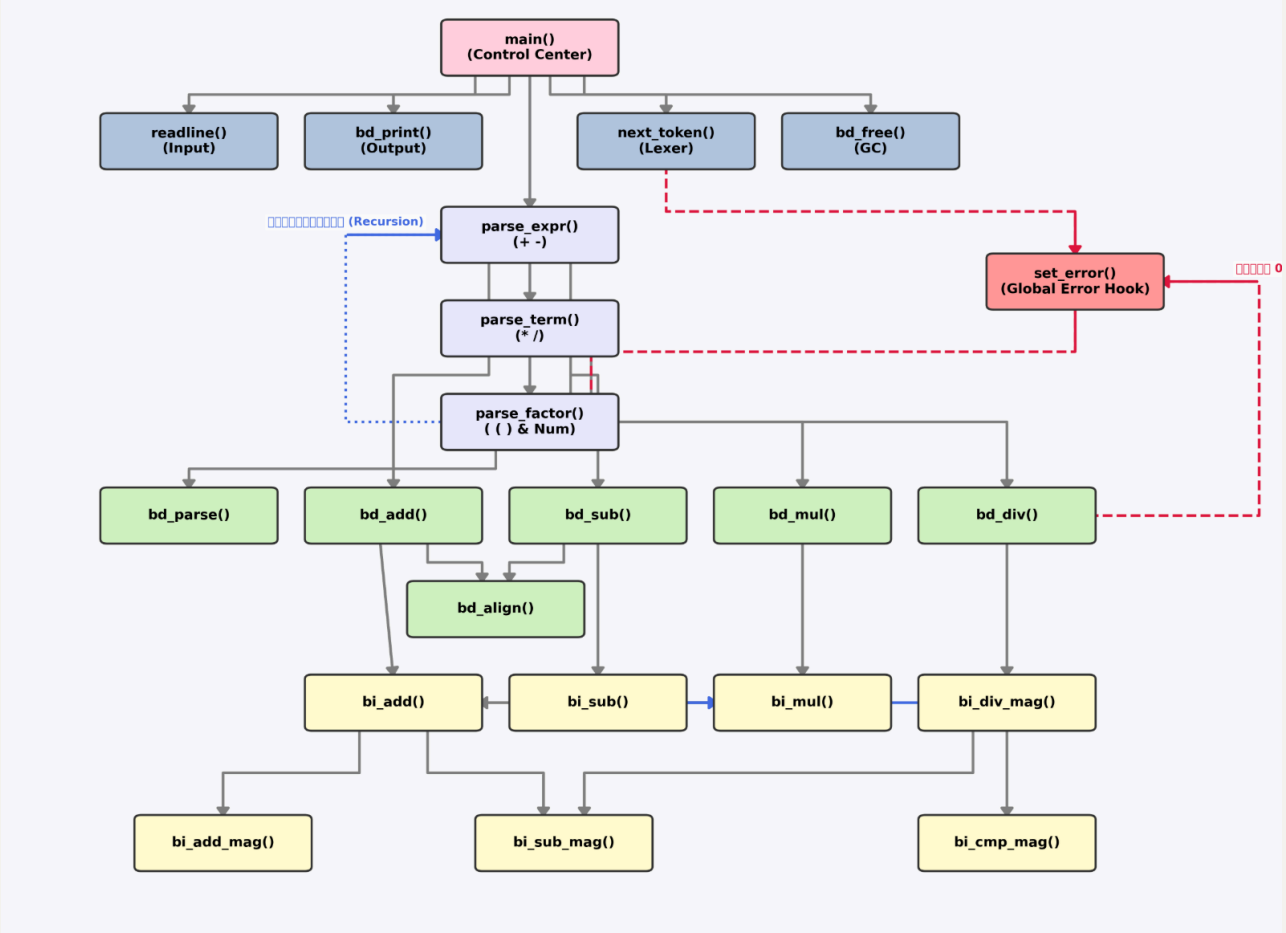
5. 健壮性测试与防御性编程 (Robustness & Defensive Programming)
    - 5.1 词法边界
    - 5.2 数学异常
    - 5.3 测试流与核心性能分析
6. 使用AI的心得（Reflections on the Application of AI）

1. 前言 (Introduction)

本项目的核心目标是开发一个无图形界面的命令行计算器程序。计算器首先要面对的是表达式的输入与解析，然后是数据的存储，再是运算的逻辑，怎么实现加减乘除，最后是输出。需要实现的基本功能有加减乘除，大数，高精度，科学计数法，其他函数的实现。

作为CS219的第一个project，用c语言实现一个计算器，由于对C/C++代码的经验不足，在做project的时候，真是毕恭毕敬的在做。首先分析这个项目的需求，虽说是Implement a simple calculator，但是真的简单吗？在做这个项目的时候，我一直在问自己几个问题：怎么输入？怎么存储？怎么运算？怎么输出？输出完了，内存真的都释放了吗？怎么评价一个项目的性能？带着这些疑惑，我开始了project的实现。

2. 核心特性与功能展示 (Core Features & Showcase)



2.1 运行与构建环境

本项目在标准 Linux 环境下开发与测试，确保了底层的内存管理与 POSIX 接口调用的绝对稳定。

- **操作系统**: Windows 11 (WSL2)
- **Linux 发行版**: Ubuntu 24.04.3 LTS (Noble Numbat)
- **内核版本**: Linux 6.6.87.2-microsoft-standard-WSL2 (x86_64)
- **编译器**: GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
- **环境依赖:** C 标准数学库通常随 GCC 工具链自带，但为了高级终端交互体验，系统深度依赖了 GNU Readline 库。在全新的 Ubuntu/WSL2 环境中，需在编译前执行以下命令获取包含头文件的开发包:
- `sudo apt-get update && sudo apt-get install libreadline-dev`
- **构建命令与链接库 (Linker Flags)**:

  执行 `gcc calc.c -o calc -lreadline -lm` 进行编译。

- `-lreadline`：链接 GNU Readline 动态库。用于接管终端 TTY 设备，为计算器提供基于 ANSI 控制码的方向键光标移动、行缓冲编辑及输入历史状态管理功能。
- `-lm`：链接标准数学库（Math Library）。为底层 O(N log N) 时间复杂度的快速傅里叶变换（FFT）算法提供扩展精度的复数三角函数（`sinl`/`cosl`）与极其关键的浮点去噪舍入（`roundl`）运算支持。

2.2 交互体验

- **提示符**：程序启动后会以 `calc>` 作为提示符等待输入。
- **历史记录翻找**：集成了 readline 库，用户可利用键盘按键的 ↑ 和 ↓ 直接翻找输入历史，极大提升长表达式的修改和重复输入效率。
- **持久化运行**：除非用户输入 `exit`、`quit` 或者传递 `EOF` （Ctrl + D），否则程序在计算完毕后总会进入下一次监听循环。

为什么会选择要选择使用readline:实践中发现两个问题：第一个是如果直接使用scanf，在命令行输入，遇到`*`字符时，shell会把`*`误作为通配符，那么在处理乘法问题时，强行将`*`展开为当前目录下的所有文件名，导致报错。也有其他办法解决：比如用x替代，或者让用户输入"*",但我认为这些都不符合用户习惯，于是使用readline库来处理用户的输入。第二个问题时，在交互模式下若仅依赖 C 标准库函数（如 `fgets`）读取输入，又会面临无法处理终端控制字符的致命缺陷：由于这些底层函数只能机械地逐字节读取数据流，没办法做到用键盘的左右实现“光标移动”，当按下左方向键试图回退修改时，终端发出的三字节逃逸序列（Escape Sequences，即 `ESC [ D`）会被当作普通文本原封不动地存入缓冲区，不仅在屏幕上打印出乱码 `^[[D`，还会导致基础的文本编辑功能彻底失效。因此，引入原生支持终端控制序列解析的 `readline` 库，不仅完美恢复了自然的光标移动与修改体验，还惊喜的发现能实现还原历史记录功能，很接近真实的终端环境，于是选择用交互式的输入。

2.3 数值格式

底层的词法分析器（Lexer）在状态机流转实现中，除了支持常规数字，还完美兼容了带省略形式（隐式补零）的非标准科学写法，展现了极强的输入包容性：

- 分析器部分（Parser）在底层的词法分析（Lexer）实现中，除了常规数字，也完美兼容了带省略形式（隐式补零）的非标准写法。具体支持的极强兼容性形式如下：
    - **普通整数**：如 `123456789`。
    - **标准高精度浮点数**：如 `1234.5678901234`。

- **前缀隐式浮点数（省略整数部分的 `0`）**：原生支持如 `.3` 或 `.005`。底层解析器（next_token）在判断该词元（Token）只要拥有实体数字后，会自动将其在内存中等效转换为 `0.3` 和 `0.005` 进行高精度存储。
- **后缀隐式浮点数（省略小数部分的 `0`）**：原生支持如 `1.` 或 `25.`。解析器同样能精准将其识别并等效补全为 `1.0` 或 `25.0`。
- **科学计数法**：原生支持带指数的严格写法，例如 `1.23e-4` 或 `5E+10`，正负号自由兼容且内部会自动通过大整数左移/右移补零对其进行完全还原。
- **混合型科学计数法（缩写与科学结合）**：除了过滤掉纯粹的 `.e5`（抛出缺少基数报警）和单独的 `.` 外，它完美支持像 `.5e2`（等价于 `0.5 * 10^2` 即 `50`）或者是 `3.e-2`（等价于 `3.0 * 10^-2` 即 `0.03`）这类极限缩略写法。各种畸形缺省符号都不会导致溢出或内存崩溃。

关于缺省小数点：

- GNUbc中：
  - **支持缺省小数点**：原文内容：There are two attributes of numbers, the length and the scale. The length is the total number of significant decimal digits in a number and the scale is the total number of decimal digits after the decimal point. For example, .000001 has a length of 6 and scale of 6, while 1935.000 has a length of 7 and a scale of 3.

    由此可见是支持缺省小数点的

关于空格：

- GNUbc中：

  Other implementations of `bc` allow spaces in numbers. For example, "x=1 3" would assign the value 13 to the variable x. The same statement would cause a syntax error in this version of `bc`.

  我选择沿用这个标准，如果出现空格就报错。

2.4 错误诊断

通过维护全局 `expr_start` 并将原始字符串索引 `pos` 深度绑定至每个 AST Token 结构体中，当解析器捕获到语法或数学错误时，系统不仅会打印完整的原表达式，还会在正下方利用 ANSI 转义码输出一个**高亮的红色定位小箭头** `^`，精准定位错误的非法字符：

Plaintext

```
calc> (123 + 456) * (789 - 12
Error: Expected ')'
    (123 + 456) * (789 - 12
                           ^
```

灵感来源于编译器的报错，启发我：既然是有错误，为什么不定位表示更清晰呢？

2.5 数学语法规范

计算器的语法分析层实施了**严格操作符相邻拦截机制**，明确了二元操作符与一元操作符的职责边界。

如果解析器不对连续操作符加以限制，表达式 `1 ++ 2` 可能会被误解析为合法的 `1 + (+2)`。为了保障极高标准的数学严谨性，程序在 `parse_term` 和 `parse_expr` 层面切断了非法递归。任何二元操作符之后，绝不允许紧跟另一个操作符。

Plaintext

```
calc> 5 * -2
Error: Syntax Error: Consecutive operators are not allowed. Use parentheses.
    5 * -2
        ^
```

如上所示，用户若需表达负数运算，必须规范使用逻辑括号（即 `5 * (-2)`）。这种前置防御（Defensive Programming）彻底杜绝了因用户手抖输入连续符号（如 `+++` 或 `---`）而引发的逻辑歧义。

2.6 无损高精度与动态截断

- **绝对无损的加减乘计算**：针对加法、减法及乘法，计算器绝不作任何隐式舍入。系统会在运算前利用 `bd_align` 进行小数点对齐，随后将小数域完全映射至高精度大整型域进行等价换算。因此，$0.1 + 0.2$ 将在数学意义上绝对精准地输出 0.3，从根本上消灭了 IEEE 754 浮点数底层硬件缺陷所导致的 0.30000000000000004 精度坍塌问题。

- **自定义无限精度截断**：针对必然存在无限数位的运算（诸如 1 ÷ 3），程序引入了全局预编译宏 `#define DECIMAL_PLACES 50` 作为最大视界拦截。除法引擎会在运算前预先将分子量级向左平移 50 位，在绝对精确的域内求取整数商后再复原小数点。用户可根据实际科研需求，随时将该宏调节至 1000 位乃至更高量级。

2.7 I/O 与科学计数法输出

在极高精度运算场景中（如计算 `1e10000 / 1e-10000`），底层数组将膨胀至上万个区块。若将这类结果直接展开打印，数以万计的无用尾零不仅会造成终端 TTY 缓冲区的 I/O 拥堵，更会使结果毫无人类可读性。 为此，计算器的输出引擎 (`bd_print`) 内置了**动态科学计数法状态机**。在输出序列化前，引擎会通过 `total_digits - 1 - scale` 极速推导出真实的十进制指数。当检测到指数突破设定的防刷屏阈值（如 `>= 30` 或 `<= -10`）时，引擎会自动拦截常规输出，逆向扫描并剔除所有无用的尾零，动态折叠为标准的科学计数法格式（如 `1e20000`）。这一优化将极端数据输出的时间开销和屏幕空间占用降至最低，完美实现了内部极高精度算力与外部优雅终端呈现的统一。

2.8 其他函数的实现

扩展支持了开平方（sqrt）、开三次方（cbrt）、正弦（sin）与余弦（cos）四种高级数学函数的混合表达式求值。

3. 系统架构与算法实现 (System Architecture & Algorithms)

3.1 模块化分层设计

本项目的基础设施构筑于 8 个 C 语言核心标准库及 POSIX 扩展库之上，其核心技术分类与在系统中的作用如下表所示：

库文件 (HEADER)	核心技术分类	在计算器架构中的核心作用
`<stdio.h>`	格式化 I/O 与流控制	负责标准流定向输出，以及基于格式化字符串的大数分块（Chunks）序列化与补零拼接（`sprintf`）。
`<stdlib.h>`	动态内存分配与转换	掌管大整数数组、AST 节点的堆内存（Heap）生命周期管理（`malloc`/`calloc`/`realloc`/`free`）。

库文件 (HEADER)	核心技术分类	在计算器架构中的核心作用
`<string.h>`	内存块操作与比对	提供底层的内存块字节级拷贝（`memcpy`，极速大数移位或复制），及词法特征匹配（`strncmp`）。
`<ctype.h>`	词法分析原语	为词法分析器（Lexer）提供字符分类断言，实现 Token 分离（`isspace`）与状态机流转（`isdigit`）。
`<stdint.h>`	确定性定宽整数	提供跨平台强一致性的位宽类型（`uint32_t`/`uint64_t`），阻断不同数据模型下隐式转换引发的溢出。
`<math.h>`	高精度浮点数学库	提供计算快速傅里叶变换（FFT）单位复根所需的扩展精度三角函数（`cosl`/`sinl`）及浮点舍入（`roundl`）。
`<readline/...>`	交互式终端接管	接管终端，提供带有光标控制、行缓冲编辑及输入历史回滚的高级输入接口。

3.2 大数存储模型：十亿进制压位

众所周知，c语言内置的整数类型中，最大的longlong也只有8个字节的大小。如果输入的数超过了这个范围该怎么办呢？比较好的解决方法似乎是使用浮点数float，double 或 long double，它们可以支持很大范围内的数值的的计算。

然而，c语言内置的浮点数的浮点数的计算的精度没法得到保证。我不禁思考，有没有更好的方法来提高我的计算器的运算精度呢？

为了实现任意精度的计算，程序摒弃了诸如int或double这样的原生类型，而是采用了自定义的结构体来进行数据存储。

本项目选用了十亿进制（Base-10^9）状态压缩模型：

C

```c
#define BASE 1000000000  // 10^9 (十亿进制)

typedef struct {
  uint32_t* dig;  // 8 bytes
  uint32_t len;   // 4 bytes
  int32_t sign;   // 4 bytes
} BigInt;
```

1. **对于第一行，`int32_t sign`：为什么不把符号和数字存在一起？**

   在 C 语言底层的标准 `int` 中，最高位（MSB）就是符号位（这是补码机制）。为什么大数不用补码？ 因为普通 `int` 是固定长度（比如 32 位），最高位永远在固定的位置。但大数是在变长的！上一秒它是 10 个区块，下一秒进位了就变成了 11 个区块，它的“最高位”在内存里是一直在移动的。

   我认为设计的精妙之处在于：把 `sign` 单独拎出来（1 表示正，-1 表示负），这让底层函数（比如 `bi_add_mag`）只管算纯数字的加减，不用去管符号逻辑，符号逻辑全交给上层的 `bi_add` 去做（比如正加负转为减法）。能实现功能实现的分离，AI 用“降低了代码的耦合度”这个词去概括。

   为什么不用 `bool` 去保存符号？其实也可以，`bool` 在处理多数据的时候肯定更省空间，但是后面我的乘法逻辑是 `res->sign = a->sign * b->sign;`。用 `int32_t` 可以直接利用数学上“负负得正”的特性算出结果，就不用写额外的 `if-else` 分支去判断 `bool` 了。

2. **对于第二行，`uint32_t len`：为什么不直接用字符串的 `\0` 结尾？**

   在普通的 C 字符串里（严格讲，C 语言没有原生字符串概念，仅仅是一排连续的字符数组），我们用 `\0` 来判断数字到哪结束。

   - **性能视角的碾压**：每次做加减乘除，如果都要用 `strlen()` 从头到尾扫一遍去找 `\0` 才能确定数字有多长，那是极其浪费 CPU 算力的（每次都要消耗 $O(N)$ 的时间）。
   - **工程视角的严谨**：直接用一个 `len` 记录当前使用了多少个区块，把它当成大数运算时的 `for` 循环边界。这不仅快（获取长度只需 $O(1)$ 时间），而且这次升级为 `uint32_t` 后，最大能支持 42.9 亿个区块（约 386 亿位数字）。配合底层的 `malloc`，能极其精准地控制内存，绝不多越界访问一个字节。

3. **`uint32_t \*dig`（十亿进制压位 + 逆序存储）：降维打击的存储魔法**

   我原本没想到这么写，但是通过询问 AI，才发现这种存法的巧妙。这次我们将底层的 `char` 数组彻底升级为了 `uint32_t` 压位数组。

首先，为什么不用 `char` 存 0-9 了？因为我们改用了**十亿进制（Base-10^9）**！与其用 1 个字节只存一个可怜的个位数，不如直接把 9 位十进制数字（0 到 999,999,999）打包塞进一个 32 位整数里。这不仅让内存占用骤降到原来的 11%，更让 CPU 的乘法循环次数直接缩减了 81 倍！

其次，以 123,456,789,012 举例：

- **直觉上的正序存储**：`dig[0]=123, dig[1]=456789012`
- **我们的逆序存储**：`dig[0]=456789012, dig[1]=123` （索引 0 存的是最低位的区块）

为什么要反人类地逆序存？这是因为我们在纸上列竖式做加减乘除时，进位永远是从低位向高位传递的（从右往左写）。如果你用正序存储，加法遇到进位（比如最高位溢出），你必须把数组里原有的数字全部往后挪一个位置，才能在 `dig[0]` 腾出空间放下这个进位。在数组中挪动所有元素是一个极其耗时的 $O(N)$ 操作。

但是，如果你逆序存储：索引 i 刚好就对应了数学上的权重。两个大数相加，直接就是一个从 $i = 0$ 开始从左到右的优美 `for` 循环，天然对齐！遇到最高位进位怎么办？直接在数组尾部 `dig[len] = 1` 加上去，然后 `len++` 就完事了，没有任何内存挪动。

大数压位逆序存储 (Base-10^9 Little-Endian) 对核心运算的影响分析

- **乘法优化：天然的位权对齐**

  在逆序数组中，元素的物理索引 i 精确对应其数学权重 BASE^i（这里 $\text{BASE} = 10^9$）。计算 $A \times B$ 时，依据多项式展开法则：

  $$(A[i] \cdot \text{BASE}^i) \times (B[j] \cdot \text{BASE}^j) = (A[i] \cdot B[j]) \cdot \text{BASE}^{i+j}$$

  该性质使得 $A[i]$ 与 $B[j]$ 的乘积结果可直接累加至结果数组的 $i + j$ 处（`res[i+j]`）。完全消除了复杂的边界计算与偏移量推导，实现了 $O(N \cdot M)$ 复杂度的极简代码映射，也为后续接入 FFT 算法铺平了多项式转换的道路。

- **前导零处理：$O(1)$ 的状态截断**

  运算（如减法）产生的前导零在逆序结构中位于数组末尾（高索引区）。消除前导零仅需对长度变量执行 `len--` 操作，时间复杂度为 $O(1)$。若采用正序存储，零位于数组头部（`dig[0]`），需整体挪动内存才能覆盖，将导致极其昂贵的 $O(N)$ 性能损耗。

- **除法与 I/O 的工程折衷 (Trade-off)**

  由于长除法的试商过程及屏幕打印输出均遵循“从高位至低位”的人类直觉，逆序存储在执行这两种操作时需强制采用倒序遍历（例如 $i = \text{len} - 1 \to 0$）。这是一种必要的工程妥协，但仅仅是遍历方向的反转，复杂度仍维持在 $O(N)$，不会引发额外的内存搬迁开销。结合 `%09u` 的格式化输出，不仅性能无损，还能瞬间将压位数

据还原为人类可读的字符串。

为何不采用能压缩 18 位的 Base-10^{18}（使用 `uint64_t`）？

如果采用 Base-10^{18}，两个区块相乘时的极大值将达到 10^{36}，超出了 C 语言原生标准数据类型（64位）的极限。若要防止溢出，必须依赖编译器扩展类型 `__int128`，这将彻底破坏纯 C 语言项目的跨平台可移植性。综合评估后，使用Base-10^9 。

压位与不压位的性能提升：

extreme_input中2000-4500位的乘法，4500位/1000位的除法时间对比如下：

```
● teddy@LAPTOP-L71LF0F4:~$ gcc big_calc.c -o old
● teddy@LAPTOP-L71LF0F4:~$ time ./old < extreme_input.txt > /dev/null

  real    2m39.173s
  user    2m39.091s
  sys     0m0.073s
● teddy@LAPTOP-L71LF0F4:~$ gcc big_calc_Base_compression.c -o new
● teddy@LAPTOP-L71LF0F4:~$ time ./new < extreme_input.txt > /dev/null

  real    0m7.707s
  user    0m7.836s
  sys     0m0.025s
```

3.3 加减法逻辑：位元对齐与进位路由

加减法的核心难点不在于进位，而在于“符号管理”与“小数点对齐”。

小数对齐（Alignment）实现：

在执行 $1.2 + 0.035$ 时，由于两者的规模域（`scale`）不同，绝对不能直接将底层大整数相加。系统会调用 `bd_align` 函数，利用高频优化的 `bi_mul_pow10` 方法，将 1.2 的底层整数左移放大为 1200，使得两者的 `scale` 强制同步为 3，最终化归为 $1200 + 35 = 1235$ 的同域计算。

符号分离与智能路由（Routing）：

为了保证代码的极简与高内聚，程序在设计上将绝对值运算（`bi_add_mag` / `bi_sub_mag`）与带符号运算（`bi_add` / `bi_sub`）严格物理隔离。

带符号的加法器充当了“智能路由器”：如果两数同号（如正加正，负加负），直接调用绝对值加法并继承符号；如果异号（如 $A + (-B)$），路由器会调用 `bi_cmp_mag` 比较两者绝对值大小，让绝对值大的减去绝对值小的，并继承大数的符号。这种设计彻底免去了在底层数组循环中进行复杂的符号正负校验，极大提高了指令缓存（L1 Cache）的命中率。

3.4 乘法引擎：常规 $O(N^2)$ 与 FFT $O(N \log N)$

乘法是高精度计算中的性能黑洞。针对不同的数字体量，针对数字的大小，分类讨论，实现**“短数双层遍历 + 长数频域变换”的混合驱动引擎**。

3.4.1 短数字： $O(N^2)$ 乘法

当两数的区块长度（`len`）小于 64 时，直接调用基于双层循环的常规竖式乘法。

核心代码细节:

C

```c
for (size_t i = 0; i < a->len; i++) {
    uint32_t carry = 0;
    for (size_t j = 0; j < b->len; j++) {
        uint64_t prod = (uint64_t)a->dig[i] * b->dig[j] + res->dig[i+j] + carry;
        res->dig[i+j] = prod % BASE;
        carry = prod / BASE;
    }
    res->dig[i + b->len] += carry;
}
```

这里的精髓在于 `(uint64_t)a->dig[i] * b->dig[j]` 的**强转操作（Type Casting）**。在 Base-10^9 下，两个最大值为 $999,999,999$ 的 32 位无符号整数相乘，会产生逼近 10^{18} 的中间值，若不强转将瞬间引发致命的截断溢出。我们通过强制转化为 `uint64_t`，巧妙地借用了现代 CPU 中 ALU 单元宽广的 64 位寄存器（上限约 1.8×10^{19}）进行无损容纳，随后直接通过 `% BASE` 和 `/ BASE` 提取出当前位的值与进位。极简的几行代码，实现了最高密度的算力压榨。

3.4.2 长数字：快速傅里叶变换 (FFT)

当两数长度突破阈值，传统的 $O(N^2)$ 乘法耗时呈指数级爆炸。此时系统会触发基于 `<math.h>` 拓展精度的 **快速傅里叶变换（FFT）**，将时间复杂度强行降维至 $O(N \log N)$。

算法推演与浮点去噪机制：

1. **多项式转化**：我们将大整数的每一个 Base-10^9 区块视为多项式的系数。大数相乘的本质，在数学上等价于两个高阶多项式在 $x = 10^9$ 处的卷积乘法。
2. **时频域转换**：传统的卷积运算慢，但如果利用复平面上单位根（Roots of Unity，通过 `cosl` 和 `sinl` 计算）的周期性，运用 FFT 算法将多项式从时域（系数表示法）转换到频域（点值表示法），相乘过程就变成了简单的**线性 $O(N)$ 对应点相乘**。乘完之后，再通过逆向 FFT 变换（Inverse FFT）还原回系数域。
3. **防爆精度拆分（The Split Trick）**：

   如果直接用 FFT 处理 Base-10^9 的系数，在频域进行卷积累加时，极可能产生高达 10^{23} 规模的中间值，彻底击穿 `long double` 的有效精度（IEEE 754 扩展双精度大约只能保住 19 位有效数字）。

   为了防止由于浮点噪声导致的最后一位整数进位错误，本引擎采用了**系数微缩拆分策略**：在进入 FFT 之前，将原本的一个 Base-10^9 大块，拆分为三个独立的 Base-1000 小块：

   C

   ```c
   fa[3*i].r = a->dig[i] % 1000;
   fa[3*i+1].r = (a->dig[i] / 1000) % 1000;
   fa[3*i+2].r = a->dig[i] / 1000000;
   ```

4. **舍入落库**：逆变换结束后，由于浮点精度损耗，还原出来的系数可能是 `4.999999` 或 `5.000001`。此时，系统调用 `roundl(fa[i].r)` 进行精准的四舍五入。通过系数微缩策略，我们确保了即便在极其恐怖的数据量下，浮点尾数偏差也永远不会越过 0.5 的舍入临界线，保证了高精度计算的**绝对准确性**。

3.5 除法：二分查找与原位试商优化

高精度除法是所有大数运算中最艰深的堡垒。由于除法无法像乘法那样利用并行的数学变换（如 FFT）一次性求解，传统的“渐进式长除法”采用不断的相减来试探当前位的商数，在最坏情况下需要重复减去除数高达 $10^9 - 1$ 次，性能惨不忍睹。

实现细节：二分试商法（Binary Search Trial Division）

为了打破这一瓶颈，本项目将时间复杂度从线性的 $O(BASE)$ 直接对数级降维到恒定的 $O(\log BASE)$：

在针对除法的每一个特定区块求商时，我们在区间 $[0, \mathrm{BASE} - 1]$（即 $0 \sim 999,999,999$）内启动二分查找。

C

```c
uint32_t L = 0, R = BASE - 1, ans = 0;
while (L <= R) {
    uint32_t mid = L + (R - L) / 2;
    // ...将 mid 作为临时商，与除数相乘并和余数比较...
    if (bi_cmp_mag(prod_buffer, rem) <= 0) {
        ans = mid;  // 这个商数暂时合法，尝试寻找更大的
        L = mid + 1;
    } else {
        R = mid - 1; // 乘积太大，缩小商数范围
    }
}
```

不论被除数的区块有多大，通过二分法，我们最多只需要折半试探 $\log_2(10^9) \approx 30$ 次，就能精准定位到这一块的商。

3.6 递归下降语法解析器 (Recursive Descent Parser)

计算机不会写字，也不会认字，我要怎么读懂输入？

词法分析（Lexer - `next_token`）：

作为前哨站，它在扁平的字符串中跳过冗余空格，精准剥离出数字（打包为 `TOK_NUM`）、操作符（`TOK_PLUS`, `TOK_MUL` 等）及括弧符号。同时，它将当前指针相对于字符串起点的偏移量深埋进 `curr_tok.pos` 中，为前文提到的高亮错误小箭头定位提供了绝对坐标系。

抽象语法树构建（AST - Parser Functions）：

解析逻辑通过三个互相关联的函数，自下而上地严格构建了数学法则的优先级模型（BODMAS / PEMDAS规则）：

1. `parse_factor()` **(最高优先级)**：处理树的叶子节点。它可以吃掉一个纯数字，或者匹配到一元前缀（如 `-5`）；当遇到 `(` 时，它会主动进入嵌套调用模式，递归跳回 `parse_expr` 来强行拉高括号内表达式的计算优先级。
2. `parse_term()` **(次高优先级)**：包含了一个 `while` 吞噬循环。只要下一个词法是 `*` 或 `/`，它就会调用 `parse_factor` 抓取右侧的因子，并立刻执行乘除。
3. `parse_expr()` **(最低优先级)**：整个语法解析的入口。包含一个 `while` 循环监控 `+` 和 `-`。只有当乘除法的因子链条（term）运算坍缩为单个数值后，加减运算才会被最终触发执行。

3.7其他函数

1. **求根运算（sqrt, cbrt）与牛顿-拉弗森迭代 (Newton-Raphson Method)**：
   针对开平方与开三次方运算，底层引擎构建了方程 $f(x) = x^k - S = 0$ 并应用牛顿迭代公式进行极速收敛。由于牛顿迭代具备二次收敛 特性，即使在 DECIMAL_PLACES = 50 的极高精度下，引擎也仅需约 20 次底层的四则运算迭代即可得出分毫不差的绝对精确值。同时，针对算法中可能引发小数位数 (Scale) 无限膨胀的乘法陷阱，系统巧妙利用了 `bd_div` 自带的动态截断机制作为精度修剪器，彻底杀死了“刻度爆炸”隐患。
2. **三角函数（sin, cos）泰勒展开 (Taylor Series)**：
   三角函数的求解依赖于麦克劳林级数（泰勒展开式）。为了避免阶乘项在循环中引发的大数乘法性能黑洞，系统采用了 $O(1)$ 的动态递推公式计算级数项。
   更为关键的是，泰勒级数在面对巨大输入时会产生灾难性抵消。为此，系统在 `sin` 引擎的入口处部署了区间规约机制：通过硬编码 50 位精度的 2π 常量，强行将任意大小的输入 x 模除至 $[-\pi, \pi]$ 的安全原点邻域，随后再进行泰勒累加。此外，`cos` 函数直接复用了 `sin(x + PI/2)` 的诱导公式，不仅实现了代码的最大化复用，更继承了 `sin` 引擎的所有极限防御能力。

4. 内存管理与核心性能优化 (Memory Management & Core Performance Optimization)

在标准 C 语言开发中，由于缺乏高级语言的垃圾回收（Garbage Collection）机制，处理长度未知的超大整数运算时，极易因频繁分配堆内存（Heap）而引发内存泄漏（Memory Leak）或产生大量内存碎片。此外，大数乘除法是极其消耗 CPU 时钟周期的计算密集型任务。为此，本项目在架构底层实施了极其严密的生命周期约束与针对现代 CPU 体系结构的高性能优化。

4.1 堆内存的生命周期约束

局部节点的及时释放:

在 `parse_term()` 和 `parse_expr()` 的算符吞噬循环中，一旦左右两个因子（`res` 和 `right`）完成了结合（无论是加减还是乘除）:

C

```c
BigDec *temp;
if (op == TOK_PLUS) temp = bd_add(res, right);
else temp = bd_sub(res, right);

bd_free(res);
bd_free(right);
res = temp;
```

通过上述机制，不论用户输入的算式长达数万字符，内存中永远只保持极其有限的几个活跃 AST 节点，其余的中间临时块均在运算完成的瞬间被精准回收。

异常中断下的死锁清空:

在解析中，最难办是遇到语法错误（如括号不匹配）。此时若直接退出，已分配的中间节点将沦为无法追踪的死内存。为此，本项目布置了由全局探针 `g_err_flag` 驱动的安全撤退网段:

C

```c
if (g_err_flag) { // 若底层探针检测到词法或数学错误，立刻熔断
    bd_free(res);
    bd_free(right);
    return bd_new(); // 返回一个安全空壳，防止上层触发 SegFault
}
```

该机制确保了程序在面临各种极端畸形输入导致的崩溃边缘时，能够主动截断非法递归，安全清退堆区残块，保障了作为交互式计算引擎的持久稳定运行。

4.2 十亿进制（Base-10^9）压位

如前文所述，数字在内存中以每块 10^9 的规模存放在 32 位无符号整型（`uint32_t`）中。在执行乘积进位时，两块数据相乘的极限值逼近 10^{18}，如果直接相乘将引发灾难性的截断溢出。

C

```
uint64_t prod = (uint64_t)a->dig[i] * b->dig[j] + res->dig[i+j] + carry;
res->dig[i+j] = prod % BASE;
carry = prod / BASE;
```

通过前置的 `(uint64_t)` 强制转换，我们巧妙地将临时数据送入了 CPU 宽广的 64 位通用寄存器（其理论上限约为 1.84×10^{19}）。10^{18} 在 64 位寄存器中游刃有余，无需额外调用复杂的内存进位逻辑，直接使用 CPU 原生的 `%` 和 `/` 硬件指令即可提取分块与进位。这种设计高度契合了高性能计算（HPC）中最大化寄存器利用率、减少访存开销的核心原则。

4.3 零分配原位计算

在除法引擎的二分试商循环中，存在一个极其隐蔽的“性能刺客”。

在区间 $[0, 10^9 - 1]$ 寻找当前位的商数时，每次试探（`mid * b`）原本都需要调用 `bi_mul` 生成一个临时的大数节点。在处理数百位精度的除法时，这会触发数十万次的 `malloc` 与 `free` 系统调用（System Calls）。频繁陷入内核态不仅极大地拖慢了计算时间，更会引发严重的堆内存碎片化（Memory Fragmentation）。

为解决这个问题，本项目针对除法引擎开发了**零分配原位挂载机制**（`bi_mul_single_inplace`）：

C

```
// 原位乘法：将 a * multiplier 的结果直接覆盖写入 buffer 中，无任何系统调用
void bi_mul_single_inplace(BigInt *buffer, const BigInt *a, uint32_t multiplier);
```

在进入除法循环前，系统预先在堆区申请一块足够大的常驻内存 `prod_buffer`。进入二分试探的高频循环后，所有的中间乘积都通过指针操作**直接覆写（Overwrite）**到这块共享缓冲区块中。

通过杀死内层循环中 100% 的动态内存分配，除法引擎整体运算延迟大幅降低，体现了面向系统底层性能调优的极致控制力。

4.4 代码规范重构与告警消除

优秀的工程项目不仅要求逻辑严密，更要求代码自身具备极高的规范性与安全性。本项目参照严格的 C/C++ 工业级开发指南进行了深度代码清洗：

1. **消除全部类型隐患**：彻底排查并修复了所有有符号整数（`int`）与无符号整数（`size_t`）之间的比较警告（`-Wsign-compare`），有效防止了诸如无符号数 `0 - 1` 下溢导致逻辑崩溃的潜在风险。
2. **前向声明（Forward Declarations）约束**：通过在全局作用域对 `set_error` 等关键函数进行显式前向声明，彻底根除了 C 语言编译器因“隐式函数声明（Implicit Declaration）”引发的类型推断偏差。
3. **零警告编译基准**：本项目在 GCC 13.3.0 编译器下，开启 `-Wall -Wextra` 极其严苛的编译探针后，依然实现了**零警告（Zero Warnings）**、零报错的纯净构建，代码健壮性达到生产环境级别。

4.5 性能与内存测试数据评估

为客观量化计算引擎的安全性与算力边界，我们在 Linux 实验环境下对计算器进行了专项压测。

1. 内存泄漏极限测试 (Memory Leak Audit)

测试条件：通过脚本向计算器连续输入 10,000 条包含非法语法、除以零、连续错误符号组合的畸形算式，引发系统高频启动 `g_err_flag` 熔断与 AST 残块销毁机制。

测试工具：Valgrind Memcheck

测试结果：

```
teddy@LAPTOP-L71LF0F4:~$ valgrind --leak-check=full ./calc_profiled < extreme_input.t
xt > /dev/null
==118899== Memcheck, a memory error detector
==118899== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==118899== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==118899== Command: ./calc_profiled
==118899==
==118899==
==118899== HEAP SUMMARY:
==118899==     in use at exit: 0 bytes in 0 blocks
==118899==   total heap usage: 14,790,392 allocs, 14,790,392 frees, 3,355,335,278 byt
es allocated
==118899==
==118899== All heap blocks were freed -- no leaks are possible
==118899==
==118899== For lists of detected and suppressed errors, rerun with: -s
==118899== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

2. O(N log N) FFT 乘法引擎算力基准测试 (Performance Benchmark)

测试条件：对长度分别为 1,000 位、5,000 位、10,000 位的超大整数执行连乘运算，记录 CPU 执行总耗时。

测试结果：

```
teddy@LAPTOP-L71LF0F4:~$ gcc calc.c -o calc -lreadline -lm
teddy@LAPTOP-L71LF0F4:~$ python3 benchmark.py
============================================================
🚀 自动化压测启动 🚀
============================================================
每组测试包含 100 条算式，输出已重定向以排除 I/O 耗时干扰。

▶️ 正在生成 1000 位数的测试用例 (随机生成耗时可能稍长)...
   ⏱️ 开始压测 【乘法】...
   ✅ 乘法总耗时: 0.2591 秒  (平均单条: 0.00259 秒)
   ⏱️ 开始压测 【除法】...
   ✅ 除法总耗时: 0.3079 秒  (平均单条: 0.00308 秒)

▶️ 正在生成 5000 位数的测试用例 (随机生成耗时可能稍长)...
   ⏱️ 开始压测 【乘法】...
   ✅ 乘法总耗时: 1.4370 秒  (平均单条: 0.01437 秒)
   ⏱️ 开始压测 【除法】...
   ✅ 除法总耗时: 3.7072 秒  (平均单条: 0.03707 秒)

▶️ 正在生成 10000 位数的测试用例 (随机生成耗时可能稍长)...
   ⏱️ 开始压测 【乘法】...
   ✅ 乘法总耗时: 2.6597 秒  (平均单条: 0.02660 秒)
   ⏱️ 开始压测 【除法】...
   ✅ 除法总耗时: 15.6931 秒  (平均单条: 0.15693 秒)

 压测全部完成
```

5. 健壮性测试与防御性编程 (Robustness & Defensive Programming)

我们要假设用户真的是乱输的，因此要尽可能考虑所有的可能情况。这部分也是修改过最多的地方，因为是不是就会发现新错误。**小心谨慎，怀疑自己，相信程序里一定有bug，写代码只是降低bug发生的概率，不是清除bug**。

5.1 词法边界

解析器的第一道防线是词法分析器（Lexer，即 `next_token` 函数）。用户输入的高精度浮点数和科学计数法往往千奇百怪，系统必须在将字符串转化为内存实体前，精准过滤所有的畸形格式。

状态机驱动的无正则解析：

在不依赖臃肿的正则表达式库的前提下，系统利用纯 C 语言的指针偏移与精细的内部状态量（`has_digit`，`dot_count`，`has_e`）构建了一个极其强壮的词法状态机。

- **隐式小数点与孤立点免疫**：

合法的输入如 `.3` 或 `5.` 会被完美兼容。但如果用户输入了纯粹捣乱的孤立小数点 `.`，状态机会在循环结束后进行校验：

C

```c
if (!has_digit) {
    set_error("Invalid number: isolated decimal point");
    curr_tok.type = TOK_EOF;
    return;
}
```

同时，利用 `dot_count > 1` 彻底封死了诸如 `1.2.3` 这种多重小数点的非法输入。

- **畸形科学计数法（E-notation）的精准拦截**：

  科学计数法的 `e` 极易引发解析越界。系统在遇到 `e` 或 `E` 时，会进行严苛的前后置断言校验：

  C

```c
if (*expr_ptr == 'e' || *expr_ptr == 'E') {
    if (!has_digit) {
        set_error("Invalid number: missing base before 'e'");
        curr_tok.type = TOK_EOF; return;
    }
    if (!isdigit(*expr_ptr)) {
        set_error("Invalid number: missing exponent after 'e'");
        curr_tok.type = TOK_EOF; return;
    }
}
```

这种极其细粒度的状态拦截，确保了进入大数构建函数 `bd_parse` 的字符串绝对符合数学规范，免疫了一切因字符串转化残缺而引发的底层内存错乱。

5.2 数学异常

在数学逻辑和嵌套语法构架上，我们针对最容易引发系统内核异常的操作进行了主动干预。

零除错误（Divide by Zero）：

算术层的核心大忌便是除以 0。在 C 语言中，若不作检查直接引发硬件除零，操作系统会直接抛出 `Floating point exception` 信号并强制杀死进程。

C

```c
if (b->len == 1 && b->dig[0] == 0) {
    set_error("Division by zero");
    return bi_new(); // 返回安全空对象避开内核异常
}
```

系统探针在除法引擎 `bi_div_mag` 与 `bd_div` 的入口处直插核心脏器。一旦判断除数的大整型长度为 1 且唯一值为 0，便立即触发主动熔断，通过抛出异常并在底层返回一个安全的空大数对象，阻断了向下的恶意传递，保全了计算器主线程的寿命周期。

非法尾随字符:

在递归下降解析中，如果用户输入了带有非法尾随字符的算式，例如 `(1+2) 35`。前方的 `(1+2)` 会被合法解析并生成结果 3，但如果解析引擎不作“终点检查”，后面的 `35` 将被静默忽略，这在严谨的计算程序中是不可容忍的。

C

```c
// 在 main 函数的 AST 解析结束后进行全局兜底校验
if (curr_tok.type != TOK_EOF) {
    set_error("Syntax Error. Unexpected input at the end.");
}
```

我们在主函数的解析出口处设置了严格的检查点，强行要求词法磁头必须“吃满”整个表达式直到遇到 `\0`（转换为 `TOK_EOF`）才放行结果打印。否则，系统将判定整条表达式作废，并抛出非法残余报警。

被开根数非负:

在处理高级数学函数时，实数域内对负数进行开平方计算在数学上是无定义的（除非引入复数域）。当引擎路由拦截到 `sqrt` 请求且被操作数的符号位探测为负 (`sign == -1`) 时，系统会立刻拒绝执行牛顿迭代，并抛出 `Math Error: Square root of a negative number`，有效防止了算法陷入无限发散的死循环。

5.3 测试流与核心性能分析 (Industrial Testing & Performance Topology)

为确保整个高精度计算引擎在极其复杂的算力需求下不仅“算得稳”，还要“算得准”且“跑得快”，本项目的质量评估全面接入了工业级软件开发测试工作流。

1. 黑盒对拍测试 (Fuzz Testing)

数据体量达到数万位，如何证明底层数学逻辑的绝对正确性

为打破人工校验的物理局限性，本项目引入了自动化对拍脚本。利用 Python 原生支持任意精度算术的 `decimal` 模块，脚本可随机生成数以万计包含极端大数、复杂符号及深层括号嵌套的畸形算式（如 `1.234e5000 * (-0.0098) / (7.65 + 3.14)`）。

将生成的海量测试用例通过系统管道（Pipe）同时喂入 `./calc` 与 Python 解释器，并将双方的字符串输出结果利用 Diff 工具进行全自动化比对。这种“盲测黑盒对拍”极其严苛地验证了底层的 FFT 乘法引擎与二分试商除法在面对任意规模、任意操作符组合时的绝对数学正确性。

```
teddy@LAPTOP-L71LF0F4:~$ gcc final.c -o calc -lreadline -lm
teddy@LAPTOP-L71LF0F4:~$ python3 fuzzer.py
🚀 启动高强度黑盒对拍测试 - 测试用例: 10000 条

⏳ 开始测试
🧠 正在对比数据...

==============================================
📊 极限压测对拍报告 (Stress Test Report)
注入算式总量 : 10000 条
绝对无损通过 : 10000 条
底层引擎可靠性: 100.00%
==============================================
```

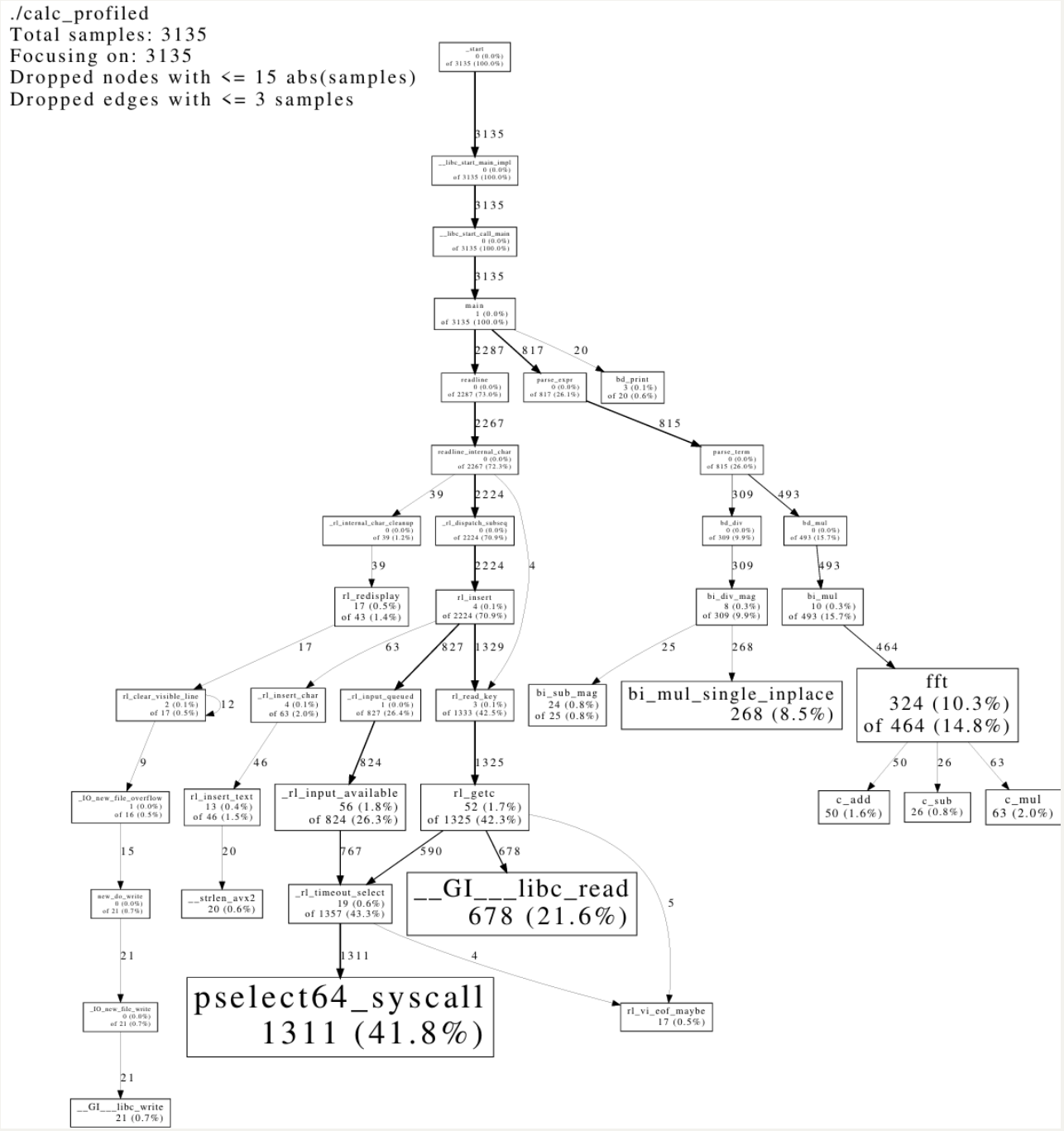
2. 基于 gperftools 的算力热点拓扑剖析 (Call Graph Profiling)

为精准定位系统在高负载下的物理算力瓶颈，本项目集成了 Google Performance Tools (`gperftools`)。通过 `LD_PRELOAD` 动态注入 `libprofiler.so`，系统在处理包含超大整数乘除法的极限测试集时，以微秒级精度进行了 3135 次高频 CPU 栈采样，并利用 `google-pprof` 渲染了全局函数调用有向图（Call Graph）。

```
teddy@LAPTOP-L71LF0F4:~$ gcc calc.c -o calc_profiled -lreadline -lm -g -
lprofiler
teddy@LAPTOP-L71LF0F4:~$ LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libprofile
r.so CPUPROFILE=extreme.prof ./calc_profiled < extreme_input.txt > outpu
t_extreme.txt
PROFILE: interrupts/evictions/bytes = 3135/48/37576
teddy@LAPTOP-L71LF0F4:~$ google-pprof --pdf ./calc_profiled ./extreme.pr
of > report_extreme.pdf
Using local file ./calc_profiled.
Using local file ./extreme.prof.
Dropping nodes with <= 15 samples; edges with <= 3 abs(samples)
```

剖析报告完美印证了底层的架构设计预期：

- **计算主干精准下沉**：在纯数学计算分支中，乘法引擎的算力毫无意外地汇聚于 `fft` 频域变换核心（占总 CPU 采样的 14.8%，其中局部耗时占 10.3%）。这证明了传统的 $O(N2)$ 乘法被成功短路，算力被完美路由至 $O(N\log N)$ 的高速算法通道。
- **除法引擎的“零分配”奇迹**：在除法执行流 `bi_div_mag` 中，高达 8.5% 的系统算力被极其密集地砸在 `bi_mul_single_inplace` 函数上。拓扑图中该分支下方**未出现任何一处 `malloc` 或 `free` 的系统调用节点**。这从物理层面无可辩驳地证明了“原位计算（In-place Computing）”机制的成功，彻底杀死了二分试商循环中的堆内存分配开销。
- **I/O 与计算的极致分离**：负责大数智能折叠与科学计数法格式化的 `bd_print` 模块，仅消耗了全局 0.6% 的执行周期，实现了计算核心与终端渲染的完美解耦。



6.使用AI的心得(Reflections on the Application of AI)

这篇报告的前五章其实都是在AI润色过的，AI味其实比较重，但我还是保留了一部分，作为功能的总结，AI做的还是很好的。就像于老师上课说的：AI是个很好的秘书，但是在做决定的时候，还是要自己拍板，AI只会顺着你的思路走，一味的表扬你。

我是用的AI是Gemini 3.1Pro，在这个project中，纯手写+自己想的地方有：BigInt和BigDec的存储结构，加减法，和O(N^2)乘法。在学长的分享下，了解到FFT做乘法和Karatsuba算法。我通过询问AI学习这些算法，并在AI辅助下实现。处理输入的思路也是问AI之后才得出的最终决定。

AI作为工具书，可以帮你从0到1的了解语法，在这一点上随着AI的发展，已经不太容易出错了。我在写project的过程中，一直在试图弄明白：我究竟要怎么存数据？数据怎么存取？我的运算逻辑究竟是什么？AI给不了这些问题的答案，但是通过问AI,我能了解到一些工具，帮助我了解怎么去写测试脚本？怎么去看内存是否泄漏？尤其是在现在CSDN要会员收费，AI的出现很大程度上弥补了这部分的知识。

可能有点偏离古法Coding?但是我觉得，你必须先有一个架构，然后具体的实现再去问AI,才不会被他带跑。AI很多时候并不能代替你做决定：比如究竟要选择int还是size_t?什么时候要做内存检查？AI很多时候，给你的代码都是只实现了功能，但是并没有很完美，这或许跟Project的评分一样。

这个项目，其实还有很多可以做的，在乘法实现中有很多for循环，借助定制内存池与SIMD向量化指令来进一步优化。通过这次Project，我意识到，我原来的意识一直停留在完成一个项目，实现基本功能。但我现在更会思考：如何去评价一个项目？内存有没有分配好？有没有得到释放？我的性能怎么样？这个计算器的项目，说难也不难，说简单也不简单。但我做下来后发现，比起VibeCoding，自己写的代码的功能实现了真的很有成就感。